

RAPPORT DE MINI-PROJET : MODÉLISATION RADIO GSM (JAVA)

1. Introduction

L'objectif de ce mini-projet est de développer un programme en Java pour représenter la partie radio du réseau GSM. Ce système gère la transmission entre l'utilisateur final (MS) et les stations de base (BTS) qui contrôlent des cellules géographiques.

2. Analyse de la Structure (Modélisation)

Le projet repose sur trois entités principales modélisées par des classes Java :

- **Classe Réseau** : Gère l'ensemble des infrastructures (nom, fréquences Uplink/Downlink, liste des BTS).

```
18 public class OnanaReseau {
19     private String nom;
20     private double freqUplink;
21     private double freqDownlink;
22     private ArrayList<OnanaBTS> listeBTS;
23
24     public OnanaReseau(String nom, double freqUplink, double freqDownlink) {
25         this.nom = nom;
26         this.freqUplink = freqUplink;
27         this.freqDownlink = freqDownlink;
28         this.listeBTS = new ArrayList<>();
29     }
30
31     public void ajouterBTS(OnanaBTS b) {
32         listeBTS.add(b);
33     }
34
35     public int calculerTotalAbonnes() {
36         int total = 0;
37         for (OnanaBTS b : listeBTS) {
38             total += b.getNbAbonnes();
39         }
40         return total;
41     }
42
43     public void rechercherLocalisation(String msisdn) {
44         for (OnanaBTS b : listeBTS) {
45             if (b.rechercherMS(msisdn) != null) {
46                 System.out.println("L'utilisateur " + msisdn + " est dans la cellule de " + b.toString());
47                 return;
48             }
49         }
50         System.out.println("Utilisateur non trouvé.");
51     }
52 }
```

- **Classe BTS** : Représente la station de base avec ses caractéristiques physiques (emplacement, milieu, capacité maximale).

```

19 public class OnanaBTS {
20     private int numero;
21     private String emplacement;
22     private String typeMilieu;
23     private int maxUtilisateurs;
24     private ArrayList<OnanaMS> utilisateursAttaches;
25
26     public OnanaBTS(int numero, String emplacement, String typeMilieu, int maxUtilisateurs) {
27         this.numero = numero;
28         this.emplacement = emplacement;
29         this.typeMilieu = typeMilieu;
30         this.maxUtilisateurs = maxUtilisateurs;
31         this.utilisateursAttaches = new ArrayList<>();
32     }
33
34     public boolean ajouterMS(OnanaMS mobile) {
35         if (utilisateursAttaches.size() < maxUtilisateurs) {
36             utilisateursAttaches.add(mobile);
37             return true;
38         } else {
39             System.out.println("BTS " + numero + " saturée !");
40             return false;
41         }
42     }
43
44     public void afficherInfos() {
45         System.out.println("BTS No: " + numero + " | Lieu: " + emplacement + " | Connectés: " + utilisateursAttaches.size());
46     }
47
48     public String toString() {
49         return "BTS No " + this.numero + " située à " + this.emplacement;
50     }
51
52     public OnanaMS rechercherMS(String msisdn) {
53         for (OnanaMS m : utilisateursAttaches) {
54             if (m.msisdn.equals(msisdn)) return m;
55         }
56     }
57 }

```

- **Classe MS** : Représente l'utilisateur (Smartphone, Tablette) avec ses identifiants (IMSI, MSISDN).

```

14 public class OnanaMS {
15     protected String nom;
16     protected String prenom;
17     private String motDePasse;
18     protected String msisdn; // Numéro de téléphone
19     protected String imsi; // Numéro de carte SIM
20     private ArrayList<String> appelsRecus;
21
22     public OnanaMS(String nom, String prenom, String motDePasse, String msisdn, String imsi) {
23         this.nom = nom;
24         this.prenom = prenom;
25         this.motDePasse = motDePasse;
26         this.msisdn = msisdn;
27         this.imsi = imsi;
28         this.appelsRecus = new ArrayList<>();
29     }
30
31     public void afficherCaracteristiques() {
32         System.out.println("MS: " + nom + " " + prenom + " | No: " + msisdn);
33     }
34
35     public void recevoirAppel(String numeroAppelant) {
36         appelsRecus.add(numeroAppelant);
37         System.out.println(msisdn + " reçoit un appel de " + numeroAppelant);
38     }
39
40     public void afficherAppels() {
41         System.out.println("Historique des appels pour " + msisdn + ": " + appelsRecus);
42     }
43
44     // Exemple d'héritage
45     class Smartphone extends OnanaMS {
46         public Smartphone(String nom, String prenom, String mdp, String msisdn, String imsi) {
47             super(nom, prenom, mdp, msisdn, imsi);
48         }
49     }
50 }

```

3. Mise en œuvre des concepts POO

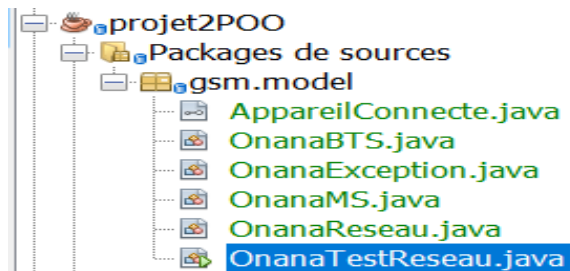
Pour répondre aux consignes, nous avons intégré les notions suivantes:

A. Encapsulation

Chaque classe utilise des attributs `private` ou `protected` pour protéger les données sensibles (comme les mots de passe des utilisateurs) et y accède via des méthodes spécifiques.

B. Héritage et Polymorphisme

Nous avons créé une hiérarchie où des classes comme `Smartphone` héritent de la classe `MS`. Cela permet de traiter n'importe quel appareil comme un objet `MS` générique tout en conservant ses spécificités.



C. Interfaces

Une interface `AppareilConnecte` a été créée pour définir un contrat de communication. Elle oblige chaque classe de type Mobile à implémenter les méthodes `emettreAppel()` et `recevoirAppel()`.

```
8  /**
9   *
10  * @author USER
11  */
12  public interface AppareilConnecte {
13
14      void emettreAppel(String destinataire);
15      void recevoirAppel(String emetteur);
16
17  }
```

4. Gestion des Exceptions

Une attention particulière a été portée à la fiabilité du système. Nous avons implémenté une exception personnalisée : `SaturationException`.

- **Fonctionnement** : Lorsqu'une BTS atteint son nombre maximal d'utilisateurs, le programme lève une exception au lieu de simplement planter.

```
12 public class OnanaException {
13     public class SaturationException extends Exception {
14         public SaturationException(String message) {
15             super(message);
16         }
17     }
18 }
19 }
```

5. Tests et Résultats

Les tests effectués dans la classe `TestReseau` confirment le bon fonctionnement du système:

1. **Ajout de BTS** : Le réseau enregistre correctement les stations.
2. **Saturation** : Le système bloque bien le 3ème utilisateur si la capacité est de 2.
3. **Localisation** : Le programme retrouve avec précision dans quelle cellule se trouve un MSISDN donné.
4. **Simulation d'appel** : L'historique des appels reçus est correctement mis à jour pour chaque utilisateur.

```
12 public class OnanaTestReseau {
13     public static void main(String[] args) {
14         try {
15
16             OnanaReseau monOrange = new OnanaReseau("Orange Radio", 900.0, 1800.0);
17             System.out.println("--- Démarrage du test Réseau ---");
18
19
20             OnanaBTS btsUrbaine = new OnanaBTS(101, "Centre-Ville", "Urbaine", 2);
21             OnanaBTS btsRurale = new OnanaBTS(202, "Village-Nord", "Rural", 10);
22
23             monOrange.ajouterBTS(btsUrbaine);
24             monOrange.ajouterBTS(btsRurale);
25
26             OnanaMS user1 = new OnanaMS("Rotto", "Ali", "pass1", "677112233", "IMSI-001");
27             OnanaMS user2 = new OnanaMS("Moussa", "Jean", "pass2", "655445566", "IMSI-002");
28             OnanaMS user3 = new OnanaMS("Zambo", "Paul", "pass3", "699001122", "IMSI-003");
29
30             try {
31                 btsUrbaine.ajouterMS(user1);
32                 btsUrbaine.ajouterMS(user2);
33
34                 btsUrbaine.ajouterMS(user3);
35
36             } catch (Exception e) {
37
38                 System.err.println("ALERTE RÉSEAU : " + e.getMessage());
39             }
40
41             System.out.println("\nTentative d'attachement à la BTS Urbaine (Max: 2)...");
42             btsUrbaine.ajouterMS(user1);
43             btsUrbaine.ajouterMS(user2);
44
45
46             // 5. Test de recherche et localisation
47             System.out.println("\n--- Vérification des données ---");
48             System.out.println("Nombre total d'abonnés inscrits : " + monOrange.calculerTotalAbonnes());
49         }
50     }
51 }
```

```

        System.out.println("\nTentative d'attachement à la BTS Urbaine (Max: 2)...");
        btsUrbaine.ajouterMS(user1);
        btsUrbaine.ajouterMS(user2);

        // 5. Test de recherche et localisation
        System.out.println("\n--- Vérification des données ---");
        System.out.println("Nombre total d'abonnés inscrits : " + monOrange.calculerTotalAbonnes());
        monOrange.rechercherLocalisation("655445566");

        // 6. Test de communication (Simulation d'appel)
        System.out.println("\n--- Simulation Appel ---");
        user2.recevoirAppel(user1.msisdn);
        user2.afficherAppels();

    } catch (Exception e) {
        System.out.println("Erreur durant l'exécution : " + e.getMessage());
    }
}

```

```

run:
--- Démarrage du test Réseau ---
BTS 101 saturée !

Tentative d'attachement à la BTS Urbaine (Max: 2)...
BTS 101 saturée !
BTS 101 saturée !

--- Vérification des données ---
Nombre total d'abonnés inscrits : 2
L'utilisateur 655445566 est dans la cellule de BTS No 101 située à Centre-Ville

--- Simulation Appel ---
655445566 reçoit un appel de 677112233
Historique des appels pour 655445566: [677112233]
BUILD SUCCESSFUL (total time: 0 seconds)

```

6. Conclusion

Ce projet nous a permis de manipuler les piliers de la POO en Java (Héritage, Polymorphisme, Interfaces, Exceptions) à travers une application concrète du réseau GSM. Le programme est optimisé, modulaire et prêt pour des extensions futures.